

design all the more important, since brute force can never investigate all possible solutions of the hardest problems. Combinatorial explosion means that we have to consider algorithmic solutions to the TSP because we don't have enough CPUs in the whole world to calculate a brute force solution.

The Nearest Neighbor Algorithm

Next we'll consider a simple, intuitive method called the *nearest neighbor* algorithm. We start with the first city on the list. Then we simply find the closest unvisited city to the first city and visit that city second. At every step, we simply look at where we are and choose the closest unvisited city as the next city on our itinerary. This minimizes the travel distance at each step, although it may not minimize the total travel distance. Note that rather than looking at every possible itinerary, as we would in a brute force search, we find only the nearest neighbor at each step. This gives us a runtime that's very fast even for very large N .

Implementing Nearest Neighbor Search

We'll start by writing a function that can find the nearest neighbor of any given city. Suppose that we have a point called `point` and a list of cities called `cities`. The distance between `point` and the j th element of `cities` is given by the following Pythagorean-style formula:

```
point = [0.5,0.5]
j = 10
distance = math.sqrt((point[0] - cities[j][0])**2 + (point[1] - cities[j][1])**2)
```

If we want to find which element of `cities` is closest to our `point` (the `point`'s nearest neighbor), we need to iterate over every element of `cities` and check the distance between the `point` and every city, as in [Listing 6-1](#).

```
def findnearest(cities,idx,nnitinerary):
    point = cities[idx]
    mindistance = float('inf')
    minidx = - 1
    for j in range(0,len(cities)):
        distance = math.sqrt((point[0] - cities[j][0])**2 + (point[1] - cities[j]
[1])**2)
        if distance < mindistance and distance > 0 and j not in nnitinerary:
```
